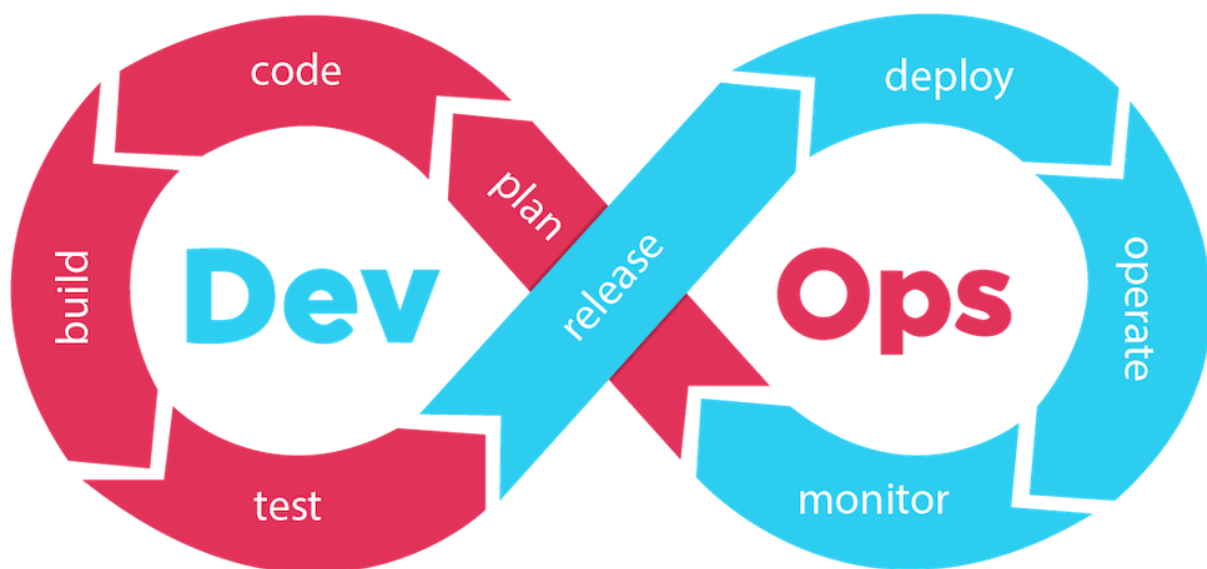




Interview Question and Answers

سوالات مصاحبه Devops قسمت هفتم



این قسمت containerization
DOCKERFILE بهینه سازی



سوال مصاحبه « ویژگی‌های یک Dockerfile بهینه چیه؟ »

مصحبه‌گر می‌تونه با لحن‌های متفاوت یا از زوایای مختلف همین سوال رو پرسه:

1. ساده و مستقیم:

- "ویژگی‌های یک Dockerfile بهینه چیه؟"
- "چطور یک Dockerfile سبک و کارآمد می‌نویسید؟"

2. مبتنی بر مشکل:

- "تصور کن Dockerfile شما خیلی سنگین شده و Buildش طولانیه. چه کارهایی انجام میدی برای بهینه‌سازی؟"

3. مقایسه‌ای:

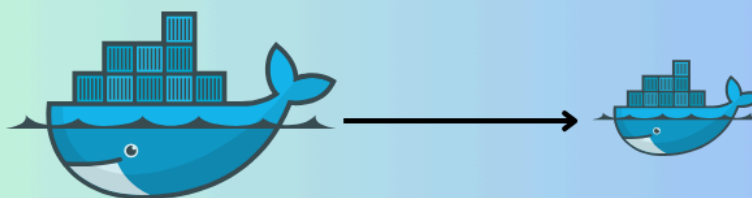
- "چه تفاوتی بین یک Dockerfile خوب و یک Dockerfile بد وجود داره؟"
- "اگه بخوای Dockerfile یه پروژه رو Refactor کنی، به چه مواردی توجه می‌کنی؟"

4. عملی/تجربی:

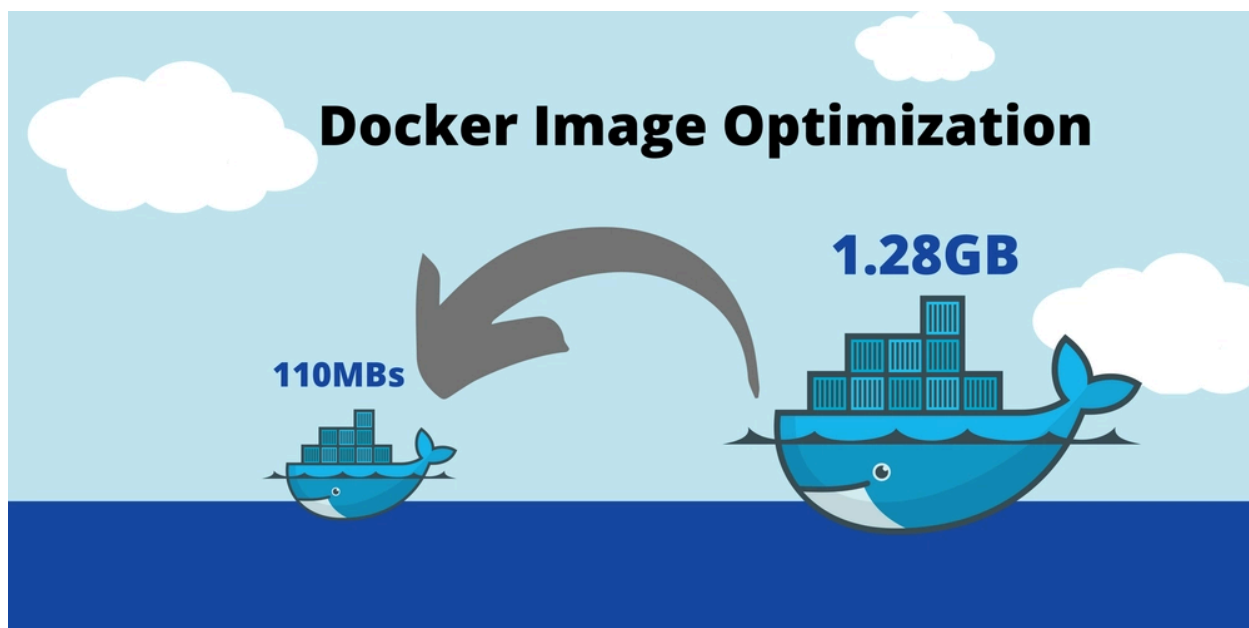
- "برای یک اپلیکیشن Python/Node.js/Java چه نکاتی در Dockerfile رعایت می‌کنی تا Image سبک‌تر بشه؟"
- "چه تکنیک‌هایی برای کاهش حجم Image استفاده می‌کنی؟"

5. جزئی‌تر و تخصصی‌تر:

- "مزایای استفاده از Multi-stage build در Dockerfile چیه؟"
- "چطور لایه‌ها (Layers) رو در Dockerfile مدیریت می‌کنی؟"



Minimize Docker Image



پاسخ کامل (ویژگی‌های Dockerfile بهینه):

یک Dockerfile بهینه باید ویژگی‌های زیر رو داشته باشه:

استفاده از Base Image متناسب:

- استفاده از **alpine** یا **slim** نسخه‌ها به جای Full image ها.

توضیح:

در هر پروژه بسته به نیاز application نسخه base image رو تعیین کنیم برای مثال راحت ترین راه برای Dockerize کردن پروژه ها اینه که یک base image ubuntu بگیریم و وابستگی ها رو روش نصب کنیم . اما واقعا راه درستی نیست . چون خیلی سنگینه و کلی چیز توش هست که نیازی بهشون نداریم مثلا برای نسخ عملیاتی بهتره از image های فوق العاده سبک Alpine استفاده کنیم .

```
FROM alpine:3.19
FROM python:3.11-alpine
FROM node:20-alpine
```

برای پروژه هایی که در محیط توسعه هستند میتونید از image های slim استفاده کنیم که حجمشون کمتر از نسخ full هست اما نسبت به alpine پکیج های بیشتری دارن .

```
FROM python:3.11-slim
FROM node:20-slim
```

مثال :

```
FROM ubuntu:22.04
RUN apt-get update && apt-get install -y python3 python3-pip
COPY . /app
WORKDIR /app
RUN pip install -r requirements.txt
CMD ["python3", "app.py"]
```

حجم container
حدوداً 400 مگابایت
بود .

با اصلاح image

برابر 60 مگابایت گردید .

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

کمینه سازی حجم Image:

- پاک کردن cache و فایل های غیرضروری بعد از نصب.
 - استفاده از `no-cache--` در apk یا `rm -rf /var/lib/apt/lists` یا `*/apt-get clean`.
- مثال :

در داکر فایل به جای دو دستور جدا به شکل زیر

```
RUN apt-get update
RUN apt-get install -y curl git
RUN apt-get clean
RUN rm -rf /var/lib/apt/lists/*
```

✓ درستش اینه همه رو تو یک خط بزنی

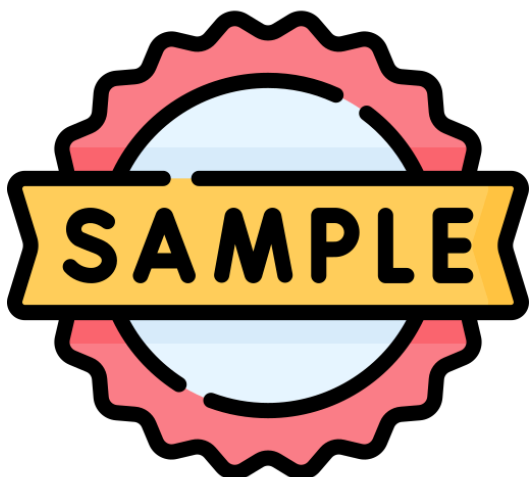
```
RUN apt-get update && apt-get install -y curl
git \
&& rm -rf /var/lib/apt/lists/*
```

یا اینکه در image های alpine
به جای

```
RUN apk update && apk add curl git
```

✓ بهتره بزنینم

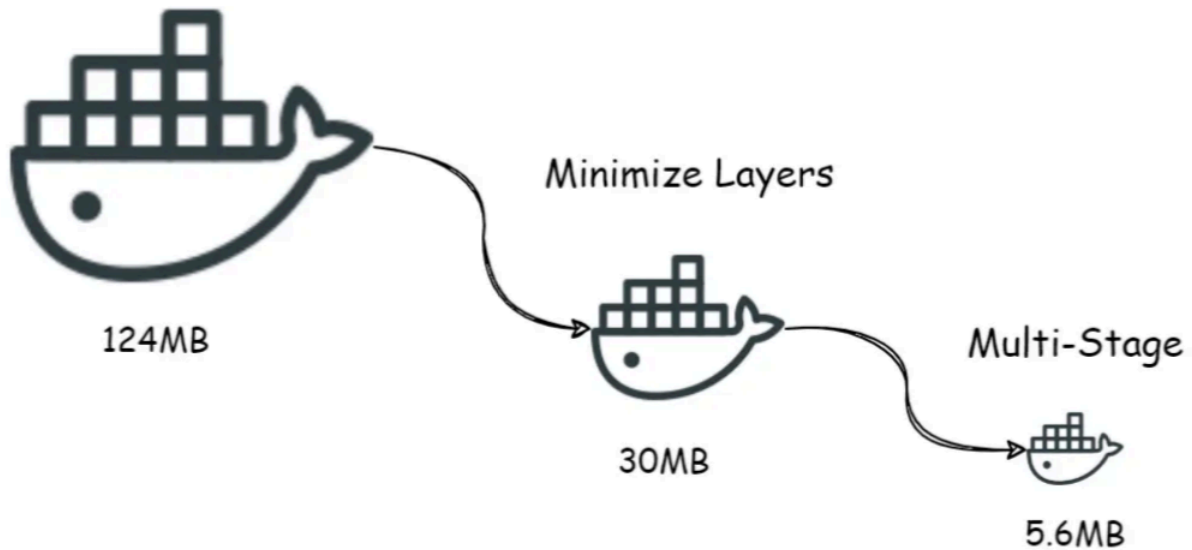
```
RUN apk add --no-cache curl git
```



:Multi-stage builds

- جدا کردن مرحله build از runtime برای حذف وابستگی‌های اضافی.

Alpine vs Distroless

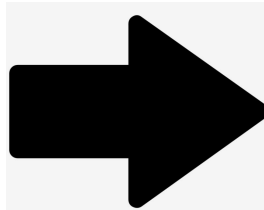


مثال در Go:

800 M

```
FROM golang:1.21-bookworm

RUN mkdir -p /home/app
WORKDIR /home/app
COPY . /home/app
RUN go mod init main.go
RUN go build
CMD ["/home/app/main.go"]
```



14 M

```
FROM golang:1.21-bookworm as builder

RUN mkdir -p /home/app
WORKDIR /home/app
COPY . /home/app
RUN go mod init main
RUN CGO_ENABLED=0 go build

FROM alpine:3.20
WORKDIR /app
COPY --from=builder /home/app /app
CMD ["/app/main"]
```

استفاده از .dockerignore:

جلوگیری از کپی شدن فایل‌های غیرضروری مثل *.log, node_modules/, git/.
.dockerignore چیه؟

ایلیه شبیه .gitignore که مشخص می‌کنه چه فایل‌ها و پوشه‌هایی نباید داخل کانتکست **Docker build** کپی بشن.
خیلی کمک میکنه که فایل‌های اضافی مثل لاگ فایل‌ها و ... نیان تو docker container

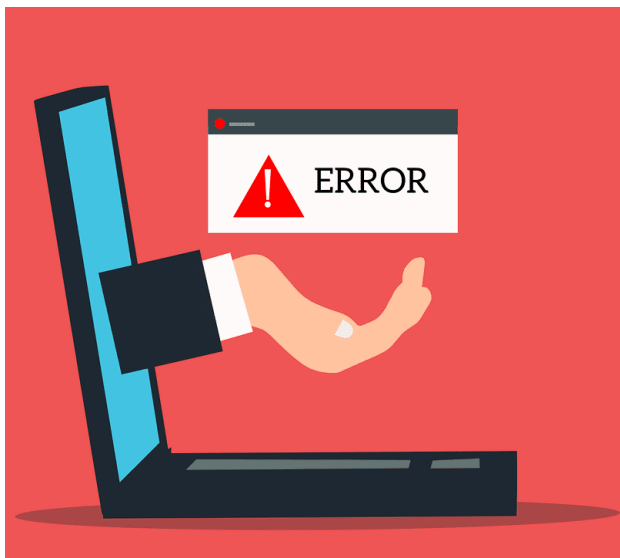
Explicit بودن نسخه‌ها:

- مشخص کردن نسخه دقیق package و base image برای جلوگیری از build های غیرقابل پیش‌بینی.

چرا Explicit بودن نسخه‌ها مهمه؟

- اگر نسخه‌ی دقیق پکیج‌ها یا Base Image رو مشخص نکنی:
 - دفعه اول Build → ممکنه درست کار کنه.
 - چند روز بعد که دوباره Build می‌کنی → خراب بشه! چون:

- نسخه جدیدی از Base image اومده.
- نسخه جدیدی از کتابخونه (مثلاً **numpy**) منتشر شده.
- وابستگی‌های پکیج‌ها تغییر کردن.



امنیت:

- اجرای اپلیکیشن با User غیر از **root**.
- به‌روزرسانی منظم پکیج‌ها.



Cache Friendly بودن:

- دستورهای مثل **COPY requirements.txt** قبل از کپی کل سورس کد تا کش بهتر کار کنه.
- ترتیب دستورات مهمه.
- وقتی Docker یک Image رو می‌سازه، هر دستور در Dockerfile → یک **لایه (Layer)** ایجاد می‌کنه. Docker برای بهینه‌سازی سرعت، این لایه‌ها رو کش (cache) می‌کنه.  یعنی:
 - اگر دستوری تغییر نکرده باشه، Docker اون لایه رو دوباره نمی‌سازه → فقط از cache استفاده می‌کنه.
 - فقط وقتی چیزی تغییر کنه، لایه‌های بعد از اون دوباره ساخته میشن.
- 👉 پس ترتیب دستورات در Dockerfile خیلی مهمه.
مثال برای Python:

```
FROM python:3.11-slim
WORKDIR /app
COPY . .
RUN pip install --no-cache-dir -r requirements.txt
CMD ["python", "app.py"]
```

1

2

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

هر دو داکر فایل به ظاهر یک کار رو انجام میدن اما

در داکر فایل 1:

- هر بار که سورس کد تغییر کنه (Docker)، **COPY . .** فکر می‌کنه requirements.txt هم تغییر کرده.

- پس کل دستور **pip install** دوباره اجرا میشه → زمان Build زیاد میشه.

در داکر فایل 2:

- اگر فقط سورس کد (**py** فایل‌ها) تغییر کنه، Docker لایه requirements.txt رو دوباره نمی‌سازه.
- فقط آخرین لایه (**COPY . .**) آپدیت میشه.
- نصب dependency ها از cache استفاده میشه → Build خیلی سریع‌تر.

📌 قانون طلایی

1. دستورهای که **خیلی کم تغییر می‌کنن** (مثل نصب dependency ها) رو **بالا** قرار بده.
2. دستورهای که **زیاد تغییر می‌کنن** (مثل سورس کد) رو **پایین** قرار بده.

Healthcheck و Entrypoint مناسب:

(این موضوع رو تو قسمت های قبلی یاد گرفتیم !!!)

- استفاده از ENTRYPOINT و CMD درست.
- تعریف HEALTHCHECK برای مانیتورینگ کانتینر.

قابلیت Debug و Maintainability:

```
# Use slim image to reduce size
FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Copy dependencies file and install
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy source code
COPY . .

# Run application
ENTRYPOINT ["python", "app.py"]
```

- داشتن کامنت های لازم.
- ساختار ساده و خوانا.

داشتن کامنت های لازم 📌

Dockerfile وقتی بزرگ میشه، فهمیدنش سخت میشه. کامنت باعث میشه:

- تیم راحت تر نگهداری کنه.
- خودت ۶ ماه بعد یادت بیاد چرا فلان دستور نوشتی 😊